Software Engineering with Generative AI

Michigan Technological University. Fall 2026 CS 5090 (3 credits, graduate)

Instructor	Dr. Jie "JW" Wu
Email	jie.jw.wu@mtu.edu
Office hours	Mon/Wed 9:30–10:30am, Rekhi 207, please make an appointment before visiting
Meeting time / room	Mon/Wed/Fri 12:00 – 12:50pm, Electrical Energy Resources, Room 0227
Semester	Aug 31 – Dec 11, 2026 (no final exam)

MTU Fall 2026 Key Dates

Per the official MTU 2026–27 academic calendar:

Date	Event
Mon, Aug 31	First day of classes (Week 1)
Fri, Sep 18	Project 2 topics circulated: the research thread starts in Week 3
Fri, Oct 2	**Project 2 research abstract due**: context, the gap in the literature, and your proposed solution, in 150–250 words
Fri, Oct 9	**Midterm 1**
Oct 19 – Nov 13	**Project 2 related-work + pitch talks**: one or two per class, at least 10 minutes each plus 1–2 minutes of Q&A: one or two related papers, then your project pitch. Sign up by Fri, Oct 16
Fri, Oct 30	**Project 1 due in full** (one deadline: all four phases + demo video)
Fri, Nov 6	**No class** (instructor travel)
Sun, Nov 8	**Project 2 proposals due**, topic pivots allowed until Fri, Nov 20
Wed, Nov 18	**Midterm 2** (covers Module 2): the latest slot MTU exam policy allows a clear run after
Fri, Dec 4	Project 2 full paper drafts due
Mon & Wed, Dec 7–9	**Project 2 design presentations + preliminary results** (in class)
Fri, Dec 11	Last day of classes; **Project 2 final paper due**
Dec 14–18	Finals week: **no final exam in this course**

Holiday class cancellations are noted in the Course Schedule below. Exact midterm days will be adjusted to the actual meeting pattern once scheduled; add/drop and withdrawal deadlines follow the Registrar's calendar.

Course Description

Generative AI (and large language models (LLMs) in particular) is materially changing how software is built. This course examines that change from both directions: **software engineering *of* generative AI systems** (building, evaluating, and operating systems with learned components) and **software engineering *with* generative AI** (AI-assisted and agentic development workflows).

AI-assisted and agentic software engineering is not a standalone unit at the end, it is a **thread that runs through the whole course**. From Week 1, students use AI coding assistants and agents as their required development workflow (and learn to validate their output); the foundations module explains how those tools work by building one; the production module covers how to engineer, evaluate, and operate LLM-powered and agentic systems. The second half of the course runs as a research seminar: students (solo or in pairs) carry out an original research project and write it up as a conference-format paper, which is submitted to a suitable venue after the semester ends.

Prerequisites

- Probability and statistics, linear algebra, data structures
- Programming maturity (Python); prior exposure to machine learning is helpful but not required
- Graduate standing, **or instructor permission for undergraduates**

Undergraduate students

This is a graduate course, and a small number of undergraduates take it with instructor permission. They do the same lectures, the same Project 1, and the same exams. Three adjustments are **recommended: suggestions, not requirements**, so that the graduate-level research expectations land more gently on a student who has not been taught them yet:

	Recommended adjustment
Project 2 pairing	Project 2 is done solo or in pairs; undergraduates are **encouraged to pair with a graduate student** rather than working solo.
Design presentation	Undergraduates paired with a graduate student may **co-present** the design presentation rather than presenting alone.

Learning Outcomes

Upon successful completion of this course, students will be able to:

1. Explain at a principled level how large language models are trained (pretraining, instruction tuning, RLHF/preference optimization) and how they generate output.
2. Build, fine-tune, and evaluate a small code-generation language model end to end, including data preparation, tokenization, training, and benchmark evaluation.
3. Analyze training datasets for provenance, bias, contamination, licensing, and governance implications.

4. Explain embeddings as a representational framework and apply them to retrieval and similarity tasks (e.g., retrieval-augmented generation over codebases).
5. Evaluate AI as a system component with defined cost, latency, reliability, safety, and privacy characteristics, and design systems that mitigate common failure modes (hallucination, adversarial inputs, miscalibration, prompt injection).
6. Apply production ML practices (requirements, architecture, deployment, testing, monitoring, and MLOps) to systems with LLM components.
7. Effectively use AI coding assistants and AI agents within a disciplined, collaborative development workflow (version control, code review, validation of AI-generated code), and design and evaluate agentic systems for SE tasks.
8. Critically read and discuss current research in AI-assisted and agentic software engineering.
9. Design and execute an empirical software engineering study involving generative AI, and communicate the results in a conference-quality research paper.

Course Materials

Two textbooks (no purchase is needed). The free materials listed below fully cover this course. T1 is a Manning book, but its companion repository (the complete, runnable code for every chapter) and the resources under it are enough on their own. **If you want to support T1's author (Sebastian Raschka, a former UW–Madison professor), you are welcome to buy the book.** T2 is freely readable online, and every other reading is a research paper available via arXiv or the ACM/IEEE digital libraries (through the MTU library).

Textbooks

#	Text	Used for
T1	S. Raschka, *Build a Large Language Model (From Scratch)*, Manning, 2024	Module 1 and Project 1: tokenization, attention, the GPT architecture, pretraining, fine-tuning, the same build-it path the course takes
T2	C. Kästner, *Machine Learning in Production: From Models to Products*, MIT Press, https://mlip-cmu.github.io/book/	Module 2: requirements, architecture, quality assurance, operations, responsible AI

T1 study materials: free, and fully sufficient for this course:

- Official companion code, chapter by chapter: https://github.com/rasbt/LLMs-from-scratch
- The author's page for the book: https://sebastianraschka.com/llms-from-scratch/
- Community study notes: https://knowledge.zhaoweiguo.com/build/html/x-learning/books/ais/2024/build_llm_from_scratch
- A community re-implementation, notebook by notebook: https://github.com/codewithdark-git/Building-LLMs-from-scratch

Additional books: free, optional

- A. E. Hassan et al., *Agentic Software Engineering: A Practical Guide for the AI-Native Engineer*, https://book.agentic-swe.dev/, the closest companion to the ⚙ thread

The ⚙ AI-assisted / agentic SE thread is grounded directly in the research literature (e.g., Hassan et al.'s agentic-SE roadmap below, and their book above) rather than an assigned textbook: the field moves faster than books do.

Research papers

A **suggested reading list**, mostly software engineering research, with the AI papers needed to make sense of it, and a few of the instructor's own: is published on the course website: **Reading list**. It is a menu, not a required set: each week's readings pair a textbook chapter with 1–2 papers drawn from it, and every slide making a specific claim shows its source at the foot of the slide.

Course Schedule

Two teaching modules with the ⚙ AI-assisted/agentic SE thread woven through both, then a flexible design-review block. Three 50-minute lectures per week (Mon/Wed/Fri) when there is no holiday. ×2 means a topic spans two sessions; ⚙ marks a practice session from the AI-assisted/agentic SE thread.

> **Sessions are subject to change.** The week-by-week content below is the plan, not a contract: pacing is adjusted to how the class is actually going, and current events in a fast-moving field sometimes earn a session. Dated deadlines and exam dates change only if announced in class *and* on the course website.
>
> **Holiday cancellations (MTU academic calendar):** no class **Mon, Sep 7** (Labor Day) (Week 2 has **two sessions**) and no classes the week of **Nov 23–27** (Thanksgiving). Marked ⛔ below.

Module 1. Foundations: Build a Code LM, with the ⚙ thread (Weeks 1–8)

Ten foundations lectures plus three ⚙ practice sessions, each ⚙ session landing just before the Project 1 phase it supports. Slides are posted on the course website after each class.

T1 (Raschka) is read in course order and **finished by Week 5, on purpose**. From Week 7 the readings are research papers, because that is where the field lives. The **one idea** column is what to keep if you keep nothing else.

Week	Dates	Sessions	The one idea	Read	Projects
1	Aug 31 – Sep 4	Course overview · ⚙ **hands-on: assistant, Git, compute, AI-use log** · what a language model is	Tokens, embeddings, next-token prediction	T1 ch. 1–2 · SLP3 ch. 6: Vector Semantics and Embeddings (PDF) · Voita: Seq2seq and Attention · 3Blue1Brown: Neural networks (videos) (both also for Week 2)	**Phase 1 released**

Week	Dates	Sessions	The one idea	Read	Projects
2	Sep 8 – 11	Attention and the Transformer block · position, norms, model families	**Attention and the Transformer block**: the core idea of the course	T1 ch. 3–4 · Vaswani et al. 2017	⛔ Labor Day: two sessions
3	Sep 14 – 18	Decoding, guided output, KV cache, MoE, prompting ×2 · ⚙ **building with AI assistance** (context engineering, decomposition, plan-then-act	Decoding and prompting) the model as an interface you tune at run time	T1 ch. 5 · Chen et al. 2021 (Codex)	training runs start · **P2 topics circulated Fri Sep 18**
4	Sep 21 – 25	Pretraining, data mix, scaling laws, SFT and LoRA ×2 · Project 1 clinic	**Pretraining and scaling**: where capability comes from; LoRA	T1 ch. 5–6 + App. E · Hoffmann et al. 2022 · Hu et al. 2021	Build pace: Sep 25 (recommended)
5	Sep 28 – Oct 2	Preferences, reward models, DPO · reasoning, test-time scaling, GRPO · ⚙ **the new workflow**: author to reviewer, levels of autonomy, the evidence	Instruction following, preferences, and reasoning models	T1 ch. 7 · Ouyang et al. 2022 · Rafailov et al. 2023 · DeepSeek-R1	**P2 abstract due Fri Oct 2**: context, gap, proposed solution
6	Oct 5 – 9	Midterm review · Phase 2 debrief · **Midterm 1 (Fri, Oct 9)**	Revise from each lecture's Key-takeaways slide	—	Use pace: Oct 9 (recommended)
7	Oct 12 – 16	Retrieval over a repo, tool calling, MCP, the agent loop ×2 · ⚙ **agentic SE** (a worked episode, tool design, SWE-bench, failure modes	**Beyond the textbook**) retrieval, tools, the agent loop	Lewis et al. 2020 · Yao et al. 2023 (ReAct) · Jimenez et al. 2024 (SWE-bench)	Evaluate & break pace: Oct 16 (recommended)
8	Oct 19 – 23	Evaluation: pass@k, benchmarks, contamination · ⚙ **testing with LLMs**, oracle problem, TDD, mutation score · module wrap	Evaluating code models honestly	Chen et al. 2021 §2.1 · one leaderboard's contamination statement	Improve pace: Oct 23 (recommended)

Midterm 1 covers the Weeks 1–6 lectures (through reasoning); the Weeks 7–8 material and the ⚙ sessions are not examined on it.

Module 2. Machine Learning in Production, with the ⚙ thread (Weeks 9–11)

Eight production lectures with three ⚙ pairings: each ⚙ session taught alongside the production topic it extends, selecting from both. Slides are posted on the course website after each class.

T2 (Kästner) is read by chapter cluster, matching the lectures; each ⚙ pairing adds one paper from the slide citations.

Week	Dates	Sessions	The one idea	Read (T2 chapters)	Projects
9	Oct 26 – 30	The model is a small box · when ML is the right tool, goals and measurement	**The model is a component**: systems thinking around a learned part	From Models to Systems · when-to-use-ML and goal-setting	**P1 due in full Fri Oct 30**
10	Nov 2 – 6	Requirements and mistakes **+ ⚙ requirements in the AI era** (paired, select) · architecture, deployment, MLOps · ⛔ **Fri: no class (instructor travel)**	Requirements, **planning for the model being wrong**; architecture	requirements + planning-for-mistakes · architecture/deployment/MLOps	**P2 proposal due Sun Nov 8**

Week	Dates	Sessions	The one idea	Read (T2 chapters)	Projects
11	Nov 9 – 13	Four-level QA **+ ⚙ assuring generated code** (paired, select) · testing in production, versioning · process, teams, debt **+ ⚙ productivity** and responsible engineering (selections)	**Quality assurance at four levels**; production; teams and responsibility	model-quality + data-quality · testing-in-production · process/teams/debt + responsible-engineering	**experiments start Mon Nov 9**: proposal feedback returned · team check-ins in office hours

Midterm 2 covers the Module 2 lectures (taught through Fri Nov 13). The ⚙ pairings are not examined.

Weeks 12–14. Midterm 2, advanced topics, then the Project 2 finish

The end of the course runs: **exam → advanced SE topics → draft → design presentation with preliminary results → final paper.** The advanced-topic sessions are flexible: they are taught from the ⚙ deep-dive material and current papers, and any of them yields to a Project 2 clinic when teams need it.

Session	Plan
Mon, Nov 16	Midterm 2 review session
Wed, Nov 18	**Midterm 2**: covers the Module 2 decks, taught through Fri Nov 13
Fri, Nov 20	⚙ **Advanced topic 1**: agentic SE, the full deep dive
⛔ Nov 23 – 27	*Thanksgiving, no classes*
Mon, Nov 30	⚙ **Advanced topic 2**: AI for testing and assurance, in depth
Wed, Dec 2	⚙ **Advanced topic 3**: instructor's pick from the current literature, or by class vote
Fri, Dec 4	Paper clinic · **draft due (end of day)**
Mon, Dec 7	**Design presentations + preliminary results**, first block (8 min + 2 questions; five talks per session)
Wed, Dec 9	**Design presentations + preliminary results**, second block
Fri, Dec 11	Course wrap: the professional stance · **final paper + artifact due**

Grading

Component	Weight
Midterm exam 1 (Module 1)	10%
Midterm exam 2 (Module 2)	10%
Participation & in-class engagement	10%

Component	Weight
Project 1. Build and evaluate a small code language model	25%
Project 2. Research project (conference submission)	45%
Bonus: workshop submission (instructor-approved)	**+5%**

There is no final exam. The two midterms are short, in-class exams: Midterm 1 covers the Module 1 foundations lectures taught through the Week 6 review (through reasoning: the Weeks 7–8 material on retrieval, agents and evaluation is *not* on it); Midterm 2 covers the Module 2 lectures. The ⚙ AI-assisted/agentic SE sessions are practice-oriented and are assessed through the projects, not an exam.

Bonus credit (+5% of the course grade). A team whose final paper is **reviewed and approved by the instructor** submits it to a suitable **conference workshop or similar venue** (e.g., an ICSE/FSE workshop, AIware, or an MSR/EASE track, chosen with the instructor over the winter break; the realistic deadlines fall January–March) and earns bonus credit for every team member. Target is a workshop paper of up to **8 pages (not including references)**. The bonus is awarded for a completed, instructor-approved submission: **not** for acceptance, which is decided long after grades are due. Approval means the paper meets a real venue's bar: a falsifiable research question, a fair baseline, reported variance, an honest threats-to-validity section, and compliance with the venue's formatting and AI-disclosure rules.

Participation & in-class engagement (10%)

- **Sign in** Attendance is required for this course, please sign the sheet each class. We will apply a threshold such as 80% in the end so that one expects to get full marks with occasional exceptions (sick, occasional conflict, etc.)
- Discussion is **assumed**: the paper discussions, the hands-on sessions and the design-presentation Q&A are how this class runs, and everyone is expected to join in naturally.
- **One way to lose points: disrespect.** Every student's questions, answers and presentations are treated with respect. Dismissing or belittling another student's contribution costs participation credit.

Project 1: Build and Evaluate a Small Code Language Model (25%, individual, Weeks 1–9)

Train a small code language model from scratch, use it on a real software engineering task, document how it fails, then make one evidence-based improvement and re-evaluate. Four phases with a recommended pace (Sep 25 / Oct 9 / Oct 16 / Oct 23); **everything is due in one bundle on Fri, Oct 30**. Full instructions, templates, compute guidance and the grading table: the Project 1 page on the course website.

Project 2: Research Study (45%, solo or in pairs, Weeks 3–14)

Carry out a small research study on generative AI in software engineering: one focused research question, an evaluation designed to test it, and a short conference-style paper (up to 8 pages, excluding references), submission-ready on **Fri, Dec 11**. Milestones run from the research abstract (Fri, Oct 2) through the presentation of related work and your pitch, the proposal, and draft to the presentation and final paper, each returned with written feedback; instructor-approved papers are submitted to a workshop or similar venue after the course for **+5% bonus credit**. Full instructions, milestone weights, idea routes, and grading: the Project 2 page on the course website, with the suggested reading list.

Course Policies

Use of AI tools

AI assistants (Copilot, Claude, ChatGPT, agents, etc.) **are permitted, subject to the conditions below.**

Conditions, in all cases:

- **You are accountable** for everything you submit. AI-generated code and text must be reviewed, tested, and understood well enough that you can defend every line without the tool in the room. "The AI wrote it" is not a defence for work that is plagiarised, incorrect, insecure, or fabricated, including invented citations, which are treated as fabrication.
- **Disclose** AI use in every deliverable: which tools, for what, and how you validated the output. Undisclosed use is an academic-integrity matter, not a style preference. It is also required to share the chat history of you and the AI.
- **Not permitted during the two in-person midterm exams.**

Communication

Email (jie.jw.wu@mtu.edu) is answered **within one business day** on weekdays. If you have heard nothing in 48 hours, send a polite nudge: the message got buried, not ignored. Questions that need more than a few lines belong in office hours (Mon/Wed 9:30-10:30, Rekhi 207, by appointment). Announcements are made in class and on the course website; the website version is authoritative.

Late work

Project milestones lose 10% per day late (max 3 days) unless arranged in advance. The Dec 4 draft and Dec 11 final deadlines are firm because the presentation sessions and the submission timeline depend on them.

Academic integrity

All work is governed by the MTU Academic Integrity Policy. Collaboration within teams is expected; representing others' (or undisclosed AI) work as your own analysis is not.

Accessibility and wellbeing

MTU complies with the ADA and Section 504. Students with disabilities should contact Student Accessibility Services and the instructor early in the semester. Standard MTU syllabus statements (accessibility, non-discrimination, Title IX, mental health resources) apply and will be linked on Canvas.
